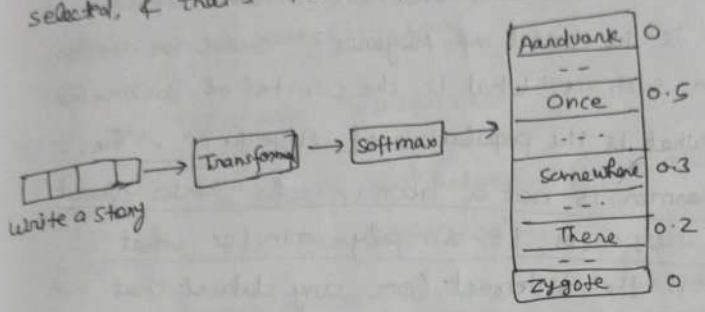
In the example below, the transformer gives the highest prob. of 0.5 to "Once" & probs of 0.3 & 0.2 to "Somewhere" & "there". Once we sample, the word "once" is selected, & that's the output of the transformer.



Now what? Well, we repeat the step. We now input the text "Write a story. Once" into the model, and most likely, the output will be "upon". Repeating this step again & again, the transformer will end up writing a story, such as "Once upon a time, there was a ...".

⇒ Post Training

Now that you know how transformers work, we still have a bit of work to do. Imagine the following: You ask the transformer "What is the capital of Algeria?" We would love for it to answer "Algiers" & move on. However, the transformer is trained on the entire internet.

The internet is a big place & it's not necessarily the best question/answer repository. Many pages, for example, would have long lists of questions w/o answers. In this case, the next sentence after "What is the capital of Algeria?" could be another question, such as "What is the capital of Burkina Faso?" or "What is the population of Algeria?". The transformer is not a human who thinks about their responses, it simply mimics what it sees on the internet (or any dataset that has been provided). So, how do we get the transformer to answer questions?

The answer is post-training. In the same way that you would teach a person to do certain tasks, you can get a transformer to perform tasks. Once a transformer is trained on the entire internet, then it is trained again on a large dataset which corresponds to lots of questions & their respective answers. Transformers (like humans), have a bias towards the last things they have learned, so post-training has proven a very useful step to help transformers succeed at the tasks they are asked to.

Post-training also helps with many other tasks. For example, one can post-train a transformer with large datasets of conversations, in order to help it perform well as a chatbot, or to help us write stories, poems or even code.

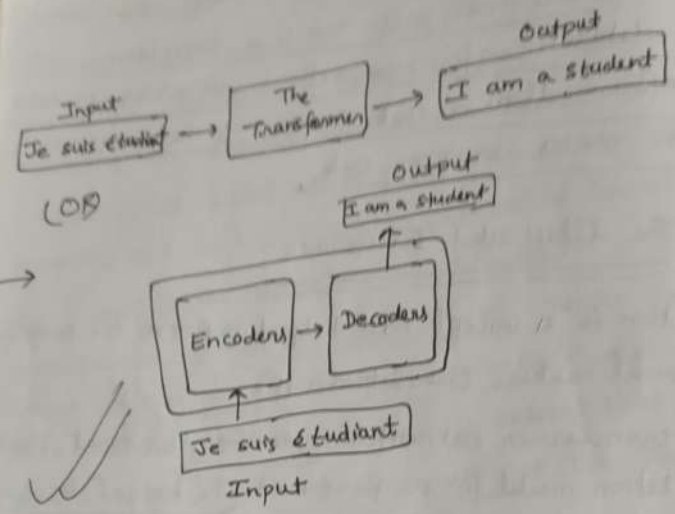
⇒ ☒ The Illustrated Transformer

Attention is a concept that helped improve the performance of neural machine translation app's.

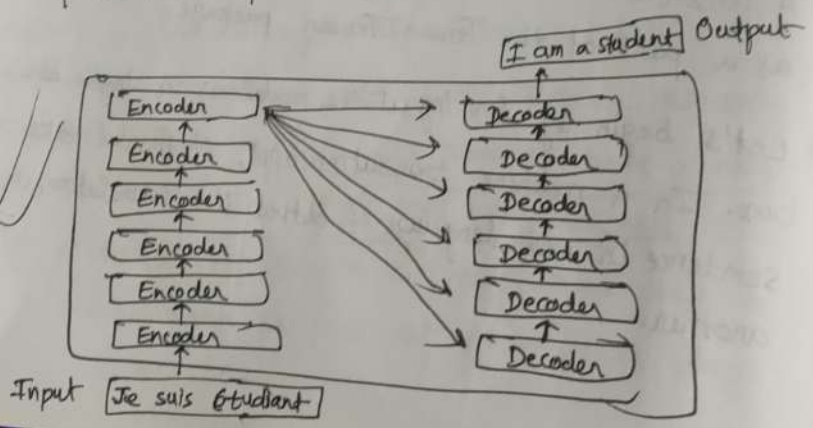
↳ The transformer outperforms the Google Neural Machine Translation model in specific tasks. The biggest benefit, however, comes from how the Transformer lends itself to parallelization. It is in fact Google Cloud's recommendation to use the Transformer as a reference model to use their Cloud TPU offering.

↳ A TensorFlow implementation of Transformer is available as a part of the Tensor2Tensor package.

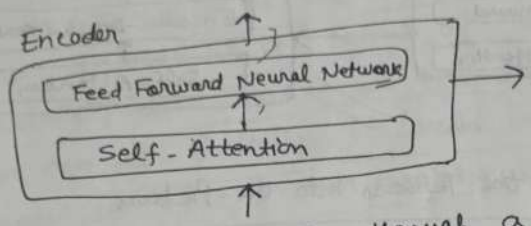
↳ Let's begin by looking at the model as a single black box. In a machine translation app^n, it would take a sentence in one language & output its translation in another.



↳ The encoding component is a stack of encoders (the paper stacks 6 of them on top of each other – there's nothing magical about the no. 6, one can definitely experiment with other arrangements.). The decoding component is a stack of decoders of the same number.



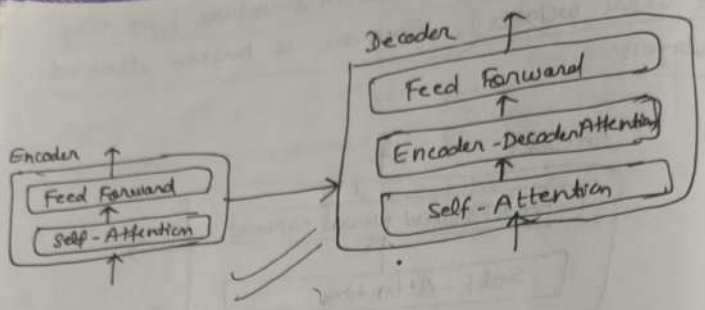
The encoders are all identical in structure (yet they do not share weights). Each one is broken down into 2 sublayers:



The encoder's inputs first flow through a self-attention layer – a layer that helps the encoder look at other words in the input ~~sequence~~ sentence as it encodes a specific word.

The outputs of the self attention layer are fed to a feed-forward neural n/w. The exact same feed-forward n/w is independently applied to each position.

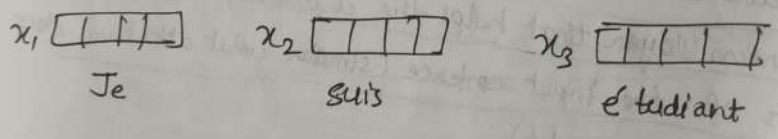
The decoder has both those layers, but b/w them is an attention layer that helps the decoder focus on relevant parts of the input sentence (similar what attention does in seq2seq models).



⇒ Bringing the Tensors into the Picture

Now that we've seen the major components of the model, let's start to look at the various vectors/tensors & how they flow b/w these components to turn the input of a trained model into an output.

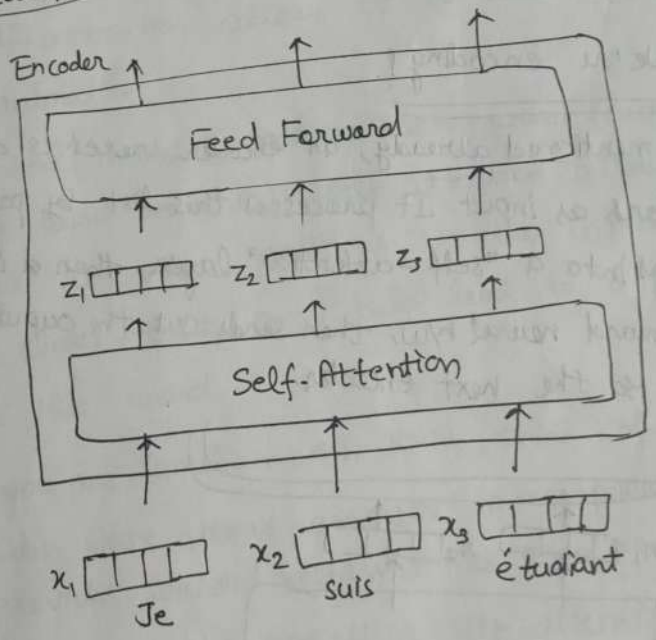
As is the case in NLP applications in general, we begin by turning each input word into a vector using an embedding algo^m.



Each word is embedded into a vector of size 512. We'll represent those vectors with these simple boxes

The embedding only happens in the bottom-most encoder. The abstraction that is common to all the encoders is that they receive a list of vectors each of size 512 – In the bottom encoder that would be the word embeddings, but in other encoders, it would be the output of the encoder that's directly below. The size of this list is hyper-parameter we can set – basically it would be the length of the longest sentence in our training dataset.

After embedding the words in our input sequence, each of them flows through each of the 2 layers of the encoder.

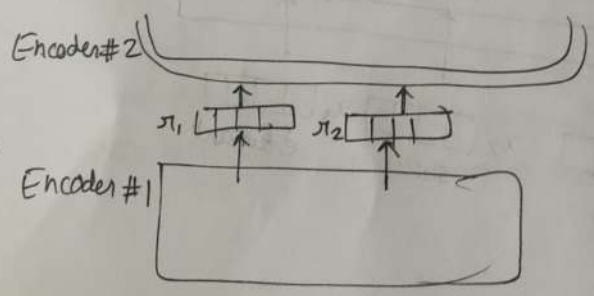


Here, we begin to see one key property of the Transformer, which is that the word in each position flows through its own path in the encoder. There are dependencies b/w these paths in the self-attention layer. The feed-forward layer does not have those dependencies, however & thus the various paths can be executed in parallel while flowing through the feed-forward layer.

↳ Next, we'll switch up the exp example to a shorter sentence & we'll look at what happens in each sub-layer of the encoder.

⇒ Now, we're encoding!

As we've mentioned already, an encoder receives a list of vectors as input. It processes this list by passing these vectors into a "self-attention" layer, then into a feed-forward neural n/w, then sends out the output upwards to the next encoder.



⇒ Self-A

Let's see

Say the fo

translate

"The ani

two ti

What doe

to the

to a h

When

self-a

"anim

As th

the

look

for C

for

If yo

a hidd

of pr

curve

meth

of

⇒ Self-Attention at a High Level

Let's see how "self-Attention" works!

Say the following sentence is an Input sentence we want to translate:

"The animal didn't cross the street because it was too tired"

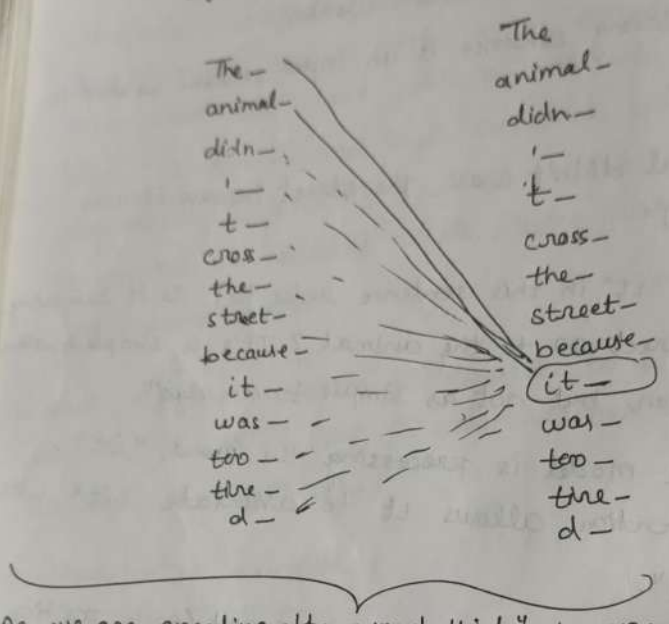
What does "it" in this sentence refer to? Is it referring to the street or to the animal? It's a simple question to a human, but not as simple to an algo^m.

When the model is processing the word, "it", self-attention allows it to associate "it" with "animal".

As the model processes each word (each position in the input sequence), self attention allows it to look at other positions in the input sequence for clues that can help lead to a better encoding for this word.

If you are familiar with RNNs, think of how maintaining a hidden state allows an RNN to incorporate its representation of previous words/ vectors it has processed with the current one it's processing. Self-attention is the method, the transformer uses to bake the "understanding" of other relevant words into the one we're currently processing.

Layer 5 Attention: Input - Input



As we are encoding the word "it" in encoder #5 (the top encoder in the stack), part of the attention mechanism was focusing on "The Animal" & baked a part of its representat^n into the encoding of "it".

[Check Tensor2Tensor notebook for visualization]